



Savitribai Phule Pune University
(Formerly University of Pune)

T.Y.B.Sc.(Computer Application)/ T.Y.B.C.A. – Semester-V

(Faculty of Science and Technology)

(For Colleges Affiliated to Savitribai Phule Pune University)

**Choice Based Credit System (CBCS) Syllabus Under
National Education Policy (NEP)**

To be implemented from Academic Year 2026-2027

Title of the Course: B.C.A./ B.Sc.(CA)



Savitribai Phule Pune University
(Formerly University of Pune)

T.Y.B.Sc.(Computer Applications)/ T.Y.B.C.A. – Semester-V

WORKBOOK

Course Code: CA – 321-VSEC
Course Title: Core Java Programming

Student Name			
College Name			
Roll No.		Division	
Exam Seat No.		Academic Year	

From the Chairman's Desk

It gives me great pleasure to present this workbook prepared by the Board of Studies in Computer Applications.

The workbook has been prepared with the objectives of bringing uniformity in the implementation of lab assignments across all affiliated colleges, acting as a ready reference for both fast and slow learners, and facilitating continuous assessment using clearly defined rubrics.

The workbook provides, for each of the assignments, the aims, prerequisites, related theoretical concepts with suitable examples wherever necessary, guidelines for the faculty/lab administrator, instructions for the students to perform assignments, and a set of exercises divided into three sets. I am thankful to the Chairman of this course and the entire team of editors. I am also thankful to the reviewers and members of BOS, **Dr. A.B. Nimbalkar and Dr. Razzak Sayyad**. I thank all members of BOS and everyone who has contributed directly or indirectly to the preparation of the workbook.

Constructive criticism is welcome and should be communicated to the Chairman of the Course and overall coordinator, **Dr. A. B. Nimbalkar**. Affiliated colleges are requested to collect feedback from the students for further improvements.

I am thankful to Hon. Vice Chancellor of Savitribai Phule Pune University **Prof. Dr. Suresh Gosavi**, Pro Vice Chancellor **Dr. Parag Kalkar**, and the Dean of Faculty of Science and Technology **Prof. Dr. Pramod D. Patil** for their support and guidance.

Prof. Dr. S. S. Sane

Chairman, BOS in Computer Applications
Savitribai Phule Pune University, Pune

Editors:

Teacher Name	College Name
Dr. Suvarna Gogate	KES' Pratibha College of Commerce and Computer Studies Chinchwad Pune
Shivarkar Sonali Yashwantrao	S. M. Joshi College, Hadapsar, Pune 28
Bagul Harshali Sachin	S. M. Joshi College, Hadapsar, Pune 28
Suvarna Sachin Pardeshi	Ahmednagar College, Ahilyanagar

Reviewed by:

• **Dr. A. B. Nimbalkar**

Assistant Professor,
PDEA's Waghire College, Pune.

• **Dr. Razzak Sayyad**

Assistant Professor,
Ahmednagar College, Ahilyanagar.

INDEX

Assignment No	Assignment Name	No. of Sessions	Page nos.
1	Basics of Java, Classes and Objects	2	8-16
2	Inheritance, Interface and package	3	17-22
3	Exception Handling, File Handling and I/O	2	23-39
4	Collections	2	40-56
5	Applet Programming and Multithreading	3	57-61

Assignment Completion Sheet

CA 321 VSEC Lab Core JAVA Programming Assignments Internal Evaluation			
Assignment No	Assignment Name	Marks (out of 5)	Teachers Sign
1	Basics of Java, Classes and Objects		
2	Inheritance, Interface and package		
3	Exception Handling, File Handling and I/O		
4	Collections		
5	Applet Programming and Multithreading		
Total (Out of 25)			
Total (Out of 15)			

Batch-Incharge

CERTIFICATE

This is to certify that **Mr./Ms.**_____

Exam Seat No. _____ have successfully completed and submitted the coursework for lab course **CA 321 VSEC Lab Core JAVA Programming, T.Y.B.Sc.(C.A.) / TYBCA Sem.V** prescribed by Savitribai Phule Pune University during Academic year-_____ and has scored marks_____ out of _____ Marks.

Batch-Incharge

Head Of the Department./Coordinator

Internal examiner

External examiner

Date:

1. About the workbook:

This LAB book / workbook is intended to be used by T.Y.B.C.A. / T.Y.B.Sc. (Computer application) students for the **CA 321 VSEC Lab Core JAVA Programming** assignments in **Semester–V**. The lab book is to be used as a hands-on resource, reference and record of assignment submission and completion by the student. The lab book contains the set of assignments which the student must complete as a part of this course.

2. The objectives of this workbook are:

- Defining the scope of the course.
- To bring the uniformity in the practical conduction and implementation in all colleges affiliated to SPPU.
- To have continuous assessment of the course and students.
- Providing ready reference for the students during practical implementation.
- Provide more options to students so that they can have good practice before facing the examination.
- Catering to the demand of slow and fast learners and accordingly providing the practice assignments to them.

3. Instructions to the students

- Students are expected to carry this book every time they come to the lab for computer science practical.
- Students should prepare oneself beforehand for the assignment by reading the relevant material.
- Instructor will specify which problems to solve in the lab during the allotted slot and student should complete them and get verified by the instructor.
- However student should spend additional hours in Lab and at home to cover as many problems as possible given in this work book.

4. Submission of Assignments:

- The Python Programming assignments are to be submitted by the student in the form of a journal containing each assignment sheet.
 - Each assignment includes the Assignment Title, Problem statement, Date of submission, Assessment date, Assessment Marks or Grade and Batch In charge's sign.
5. Assessment:
- Continuous assessment of laboratory work is to be done based on overall performance and lab assignments performance of student.
 - Each lab assignment assessment will be assigned grade / marks based on parameters with appropriate weightage.
 - Suggested parameters for overall assessment as well as each lab assignment assessment includes: timely completion, performance, innovation, efficient codes and good programming practices.
 - Students will be assessed for each exercise on a scale from 0 to 5.

Not done	0	Needs improvement	3
Incomplete	1	Complete	4
Late Complete	2	Well Done	5

Assignment No.	1	No. of Sessions:	2
Assignment Name :	Basics of Java, Classes and Objects		

Objective:

- To learn the basic concepts of Java programming.
- To write, test, and debug simple Java programs.
- To implement Java programs with basic concepts.

Reading:

- OOP's Concepts
- Keywords used in Java
- Structure of Java Program

Understanding:

Step 1: Creation of a Java program: -

Creating a Java program means writing a Java program on any editor or IDE. After creating a Java program, provide a .java extension to file. It signifies that the particular file is a Java source code. Also, whatever progress we do from writing, editing, storing and providing .java extension to a file, it basically comes under creating of a Java program.

Step 2: Compiling a Java Program

Our next step after the creation of the program is the compilation of the Java program. Generally, Java program which we have created in step 1 with a .java extension, it has been compiled by the compiler. Suppose if we take an example of a program, say Welcome.java, when we want to compile this, we use a command such as javac. After opening command prompt or shell console we compile Java program with .java extension as:

`javac Welcome.java`

After executing the javac command, if no errors occur in our program we get a .class file which contains the bytecode. It means that the compiler has successfully converted Java source code to bytecode. Generally, bytecode is used by the JVM to run a particular application. The final bytecode created is the executable file which only JVM understands. As Java compiler is invoked by javac command, the JVM is invoked by java command. On the console window you must type:

`java Welcome`

This command will invoke the JVM to take further steps for execution of the Java program.

Step 3: Program loading into memory by JVM:-

JVM requires memory to load the .class file before its execution. The process of placing a program in memory in order to run is called as Loading. There is a class loader present in the JVM whose main functionality is to load the .class file into the primary memory for the execution. All the .class files required by our program for the execution are loaded by the class loader into the memory just before the execution.

Step 4: Bytecode Verification by JVM:-

In order to maintain security of the program JVM has a bytecode verifier. After the classes are loaded into memory, a bytecode verifier comes into picture and verifies bytecode of the loaded class in order to maintain security. It checks whether bytecodes are valid. Thus it prevents our computer from malicious viruses and worms.

Step 5: Execution of Java program: -

Whatever actions we have written in our Java program, JVM executes them by interpreting bytecode. JVM uses a JIT compilation unit which we even call just-in-time compilation.

JVM (Java Virtual Machine):-

JVM, i.e., Java Virtual Machine. JVM is the engine that drives the Java code. Mostly in other Programming Languages, compilers produce code for a particular system but Java compilers produce Bytecode for a Java Virtual Machine. When we compile a Java program, then bytecode is generated. Bytecode is the source code that can be used to run on any platform. Bytecode is an intermediary language between Java source and the host system. It is the medium which compiles Java code to bytecode which gets interpreted on a different machine and hence it makes it Platform/Operating system independent.

Jdk (Java Development Kit) Tools:

Java environment includes a few development tools, classes and methods. The development tools are part of the system known as Java Development Kit (JDK) and the classes and methods are part of the Java Standard Library (JSL), also known as the Application Programming Interface (API).

A Simple Java Program

```
public class Test
{
    public static void main(String args[])
    {
```

```

        System.out.println("Hello students");
    }
}

```

class keyword is used to declare a class in Java.

public keyword is an access modifier that represents visibility. It means it is visible to all.

static is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main () method is executed by the JVM, so it doesn't require creating an object to invoke the main () method. So, it saves memory.

void is the return type of the method. It means it doesn't return any value.

main represents the starting point of the program.

System.out.println() is used to print statements.

Here, System is a class, out is an object of the PrintStream class, println() is a method of the PrintStream class.

To compile the program the command is

```
javac Test.java
```

To execute the program the command is

```
java Test
```

Different ways of reading input from console in Java -

1. Using command line argument: The java command-line argument is an argument i.e. passed at the time of running the java program. The arguments passed from the console can be received in the java program and it can be used as an input.

```

public class Test
{
    public static void main(String[] args)
    {
        int a,b,c;
        a = Integer.parseInt(args[0]);
        b = Integer.parseInt(args[1]);
        c = a+b;
        System.out.println("Addition:" +c);
    }
}

```

```
javac Test.java
java Test 10 20
```

2. Using Buffered Reader Class

This is the Java classical method to take input. This method is used by wrapping the System.in (standard input stream) in an InputStreamReader which is wrapped in a BufferedReader, we can read input from the user in the command line.

```
import java.io.*;
public class ConsoleInput
{
    public static void main (String[] args) throws IOException
    {
        int rollNumber;
        String name;
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter the roll number: ");
        rollNumber = Integer.parseInt(br.readLine());
        System.out.println(" Enter the name: ");
        name = br.readLine();
        System.out.println(" Roll Number = " + rollNumber);
        System.out.println(" Name = " + name);
    }
}
```

3. Using Scanner Class

This is probably the most preferred method to take input. The main purpose of the Scanner class is to parse primitive types and strings using regular expressions, however it also can be used to read input from the user in the command line.

```
import java.util.Scanner;
public class ScannerTest
{
    public static void main(String args[])throws Exception
    {
        Scanner sc=new Scanner(System.in);
```

```

        System.out.println("Enter your rollno and name :");
        int rollno=sc.nextInt();
        String name=sc.next();
        System.out.println("Rollno:"+rollno+" Name:"+name);
        sc.close();
    }
}

```

Class: Collection of objects is called class.

Syntax to create a Class:

A keyword class is used to create a class.

```
class Classname
```

```

{
    //Variable declaration
    //Method declaration
}

```

For Example: Number.java

```

class Number
{
    int x=10; //variable declaration
}

```

Object: Objects have states and behaviours.

It is defined as an instance of a class. An object contains an address and takes up some space in memory.

Classes create objects and objects use methods to communicate between them.

Syntax to create an Object:

```
ClassName objectName = new ClassName();
```

An object is created for a class by using the class name and new keyword.

For Example:

```
Number Obj=new Number();
```

Instance variables and methods are accessed by using objects.

```
objectname.variableName; //Accessing variable
```

objectname.methodName(); // Accessing a class method
For Example: Java Program to demonstrate Class & Object.

```
class Number
{
    int a,b;
    void disp()
    {
        System.out.println(a+ " " + b);
    }
}
class CDemo
{ // Main Class
    public static void main(String args[])
    {
        Number obj=new Number();
        obj.a=10;
        obj.b=20;
        obj.disp();
    }
}
```

Array of Objects: - An array that contains class type elements is known as an array of objects. It stores the reference variable of the object. Before creating an array of objects, we must create an instance of the class by using the new keyword. Suppose, we have created a class named Student. We want to keep marks of 20 students. In this case, we will not create 20 separate objects for the Student class. Instead of this, we will create an array of objects, as follows:

```
Student s[] = new Student[20];
```

Constructor: A constructor in Java is a special method that is used to initialize objects of the class. The constructor is called when an object of a class is created. The constructor has the same name as the class itself. Constructors have no explicit return type.

Types of Constructor:

a. Default/No-argument constructor: A constructor that has no parameter is known as default constructor.

b. Parameterized constructor: A constructor that takes parameters is known as parameterized constructor.

For Example: Java Program to demonstrate types of Constructor.

```

class Number
{
    int num;
    Number() //Default Constructor
    {
        num=10;
    }
    Number(int a) //Parameterized Constructor
    {
        num=a;
    }
    void Display()
    {
        System.out.println("num = " +num);
    }
}
class CDemo // Main Class
{
    public static void main(String args[])
    {
        Number obj1=new Number(); //invokes Default constructor
        Number obj2=new Number(50); //invokes Parameterized constructor
        obj1.Display();
        obj2.Display();
    }
}

```

this keyword:

This keyword can be used to refer to the current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

```

import java.io.*;
public class Student
{
    private int rollno;
    private String name;

```

```
private float fee;
public Student(int rollno,String name,float fee)
{
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
public void display()
{
    System.out.println(rollno+" "+name+" "+fee);
}
public static void main(String args[])
{
    Student s1=new Student(111,"ankit",5000);
    s1.display();
}
}
```

Lab Assignment

SET A

1. Write a Java program to accept a number using the BufferedReader class and generate the multiplication table of the given number.
2. Write a Java program to reverse a number. Accept the number using command line arguments.
3. Write a Java program to find the sum of elements of an array. Also display the array elements in ascending order.
4. Write a Java program to create a class MyDate with data members dd, mm, and yy. Implement default and parameterized constructors. Display the date in dd-mm-yy format using this keyword.

SET B

1. Define a class IntegerNumber with a private data member. Write methods to calculate the square and cube of the number. Accept input using command line arguments.
2. Write a Java program to define a class Book with data members bookId, bookName, and price. Store details of 5 books using an array of objects and display it.
3. Write a Java program to create a class Customer with data members cid, cname, and billAmount. Create an array of 'n' customers and sort them based on bill amount in ascending order.

SET C

1. Define a class person(pid, pname, age, gender). Define Default and parameterised constructor. Overload the constructor. Accept the 5 person details and display it.(use this keyword).

Signature of the instructor: _____ Date: _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Assignment No.	2	No. of Sessions:	3
Assignment Name :	Inheritance, Interface and package		

Objective:

- To learn the Inheritance, Interface and package in Java Programming.
- To write Java Program using Inheritance, Interface and package.
- To develop a logic with Inheritance, Interface and package.

Reading:

- Constructor/ Destructor
- Concept of Overloading and overriding

Understanding:

Inheritance

Inheritance is one of the features of Object Oriented Programming System (OOPs) it allows the child class to acquire the properties (the data members) and functionality (the member functions) of the parent class.

What is a child class?

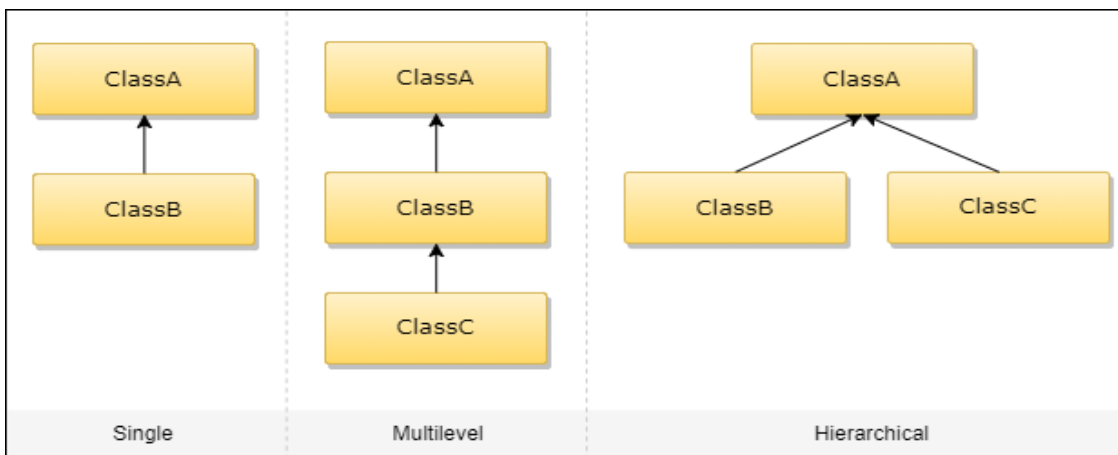
A class that inherits another class is known as child class, it is also known as derived class or subclass.

What is parent class?

The class that is being inherited by another class is known as parent class, super class or base class.

Types of inheritance:

A) Single Inheritance: One subclass is derived from only one superclass is called as single inheritance.



Java Inheritance

Access in subclass

The following members can be accessed in a subclass:

- i) public or protected superclass members.
- ii) Members with no specifier if the subclass is in the same package.

The use of super keyword

- i) Invoking superclass constructor - super(arguments)
- ii) Accessing superclass members – super.member
- iii) Invoking superclass methods – super.method(arguments)

Example:

```
public class A
{
    protected int num;
    public A(int num)
    {
        this.num = num;
    }
}
public class B extends A
{
    private int num;
    public B(int a, int b)
    {
        super(a); //should be the first line in the subclass constructor
        this.num = b;
    }
    public void display()
    {
        System.out.println("In A, num = " + super.num);
        System.out.println("In B, num = " + num);
    }
}
```

Overriding methods

Redefining superclass methods in a subclass is called overriding. The signature of the subclass method should be the same as the superclass method.

Example:

```
public class A
{
```

```

    public void display(int num)
    { //code }
}
public class B extends A
{
    public void display(int x)
    { //code }
}

```

Abstract class

An abstract class is a class which cannot be instantiated. It is only used to create subclasses. A class which has abstract methods must be declared abstract. An abstract class can have data members, constructors, method definitions and method declarations.

abstract access specifier class ClassName { ... }

Abstract method

An abstract method is a method which has no definition. The definition is provided by the subclass.

abstract access specifier returnType method(arguments);

Interface:

An interface looks like a class but it is not a class. It is a collection of only abstract methods and static final variables. Interface cannot be instantiated. An interface does not contain a constructor. Interface is used to achieve Multiple Inheritance & abstraction.

Syntax to define an interface in java:

```

interface InterfaceName
{
    ...
    //members declaration;
    //method declaration;
    ...
}

```

Syntax to implement interface:

```

class className implements InterfaceName
{
    ...
    body-of-the-class
    ...
}

```

```
}
```

“implements” keyword is used to inherit interface in interface/class.

```
public interface MyInterface
{
    int size= 10; //final and static
    void method1();
    void method2();
}
public Class MyClass implements MyInterface
{
    //define method1 and method2
}
```

Package

A java package is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Syntax:

```
package packagename; //Simple example of java package
```

The package keyword is used to create a package in java.

```
//save as Simple.java
```

```
package mypack;
public class Simple
{
    public static void main(String args[])
    {
        System.out.println("Welcome to package");
    }
}
```

How to compile java package

If you are not using any IDE, you need to follow the syntax given below:

```
javac -d directory java filename
```

For example

```
javac -d . Simple.java
```

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program -

You need to use a fully qualified name e.g. mypack.Simple etc to run the class.

Lab Assignment

SET A

1. Write a java program to calculate area of Cylinder and Circle.(Use super keyword)
2. Define an Interface Shape with abstract method area(). Write a java program to calculate an area of Circle and Sphere.(use final keyword)
3. Define an Interface “Integer” with a abstract method check().Write a Java program to check whether a given number is Positive or Negative.
4. Define a class Employee having private members-id, name, department, salary. Define default & parameterized constructors. Create a subclass called Manager with private member bonus. Define methods accept & display in both the classes. Create n objects of the manager class & display the details of the manager having the maximum total salary(salary+bonus).
5. Write a package game which will have 2 classes Indoor & Outdoor. Use a function display() to generate the list of player for the specific game. Use default & parameterized constructor.

SET B

1. Define an interface “Operation” which has methods area(),volume(). Define a constant PI having a value 3.142. Create a class circle (member – radius), cylinder (members – radius, height) which implements this interface. Calculate and display the area and volume.
2. Write a Java program to create a super class Employee (members – name, salary). Derive a sub-class as Developer (member – projectname). Derive a sub-class Programmer (member – proglanguage) from Developer. Create object of Developer and display the details of it. Implement this multilevel inheritance with appropriate constructor and

methods.

3. Define an abstract class Staff with members name and address. Define two sub-classes of this class – FullTimeStaff (members - department, salary, hra - 8% of salary, da – 5% of salary) and PartTimeStaff (members - number-of-hours, rate-per-hour). Define appropriate constructors. Write abstract method as calculateSalary() in Staff class. Implement this method in subclasses. Create n objects which could be of either FullTimeStaff or PartTimeStaff class by asking the user's choice. Display details of all FullTimeStaff objects and all PartTimeStaff objects along with their salary.

SET C

1. Derive a class Square from class Rectangle. Create one more class Circle. Create an interface with only one method called area(). Implement this interface in all classes. Include appropriate data members and constructors in all classes. Write a java program to accept details of Square, Circle & Rectangle and display the area.

Signature of the instructor: _____ **Date:** _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Assignment No.	3	No. of Sessions:	2
Assignment Name:	Exception Handling, File Handling and I/O		

Objective:

- To learn basic exception handling concept in Java Programming.
- To write Java Program using try and catch.
- To develop a logic with Inheritance, Interface and package.

Reading:

- Error handling in programming.
- File concept and file operations.
- Input streams and output stream concept.

Understanding:

Exception Handling:

Exception Handling is the mechanism to handle runtime malfunctions. We need to handle such exceptions to prevent abrupt termination of program. The term exception means exceptional condition, it is a problem that may arise during the execution of program. A bunch of things can lead to exceptions, including programmer error, hardware failures, files that need to be opened cannot be found, resource exhaustion etc.

Exception: A Java Exception is an object that describes the exception that occurs in a program. When an exceptional event occurs in java, an exception is said to be thrown. The code that's responsible for doing something about the exception is called an exception handler.

Exception class Hierarchy: All exception types are subclasses of class Throwable, which is at the top of exception class hierarchy.

Exception class is for exceptional conditions that program should catch. This class is extended to create user specific exception classes.

1. RuntimeException is a subclass of Exception. Exceptions under this class are automatically defined for programs.
2. Exceptions of type Error are used by the Java run-time system to indicate errors having to do with the run-time environment, itself. Stack overflow is an example of such an error.

Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception.

1. Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. Checked exceptions are checked at compile-time.

- Exception
- IOException
- FileNotFoundException
- ParseException
- ClassNotFoundException
- CloneNotSupportedException
- InstantiationException
- InterruptedException
- NoSuchMethodException
- NoSuchFieldException

2. Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions.

Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

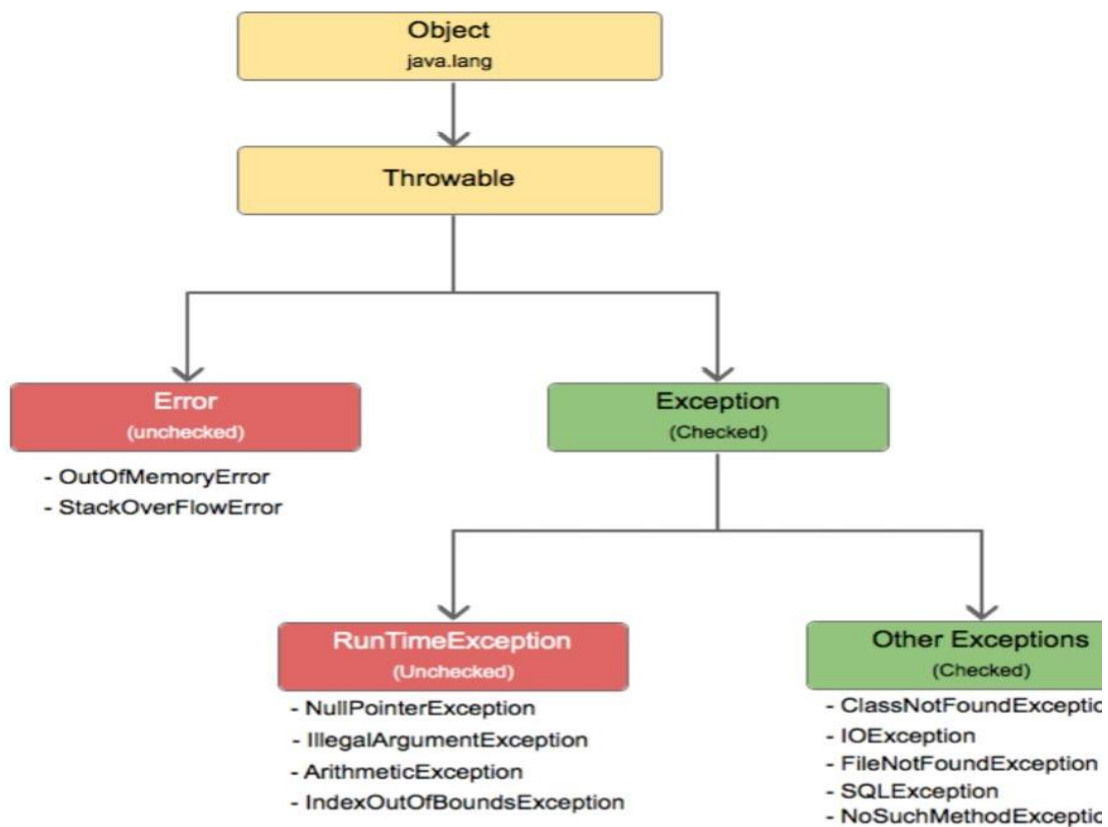
Unchecked Exceptions:

- ArrayIndexOutOfBoundsException
- ClassCastException
- IllegalArgumentException
- IllegalStateException
- NullPointerException
- ArithmeticException

3. Error

Error is irrecoverable. Some example of errors are

OutOfMemoryError, VirtualMachineError, AssertionError etc.



Exception Handling Mechanism

In java, exception handling is done using five keywords,

1. The "**try**" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
2. The "**catch**" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
3. The "**finally**" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
4. The "**throw**" keyword is used to throw an exception.
5. The "**throws**" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Exception handling is done by transferring the execution of a program to an appropriate exception handler when exception occurs.

Syntax of Java try-catch

```
try
{
//code that may throw an exception
}catch(Exception_class_Name ref){}
```

Example

```
public class TryCatchExample
{
public static void main(String[] args)
{
try
{
int data=50/0; //may throw exception
// if exception occurs, the remaining statement will not execute
System.out.println("rest of the code");
}
// handling the exception
catch(ArithmeticException e) {
System.out.println(e);
}
} }
```

Syntax of try-finally block

```
try
{
//code that may throw an exception
}finally{}
```

Example

```
public class TestFinallyBlock
{
public static void main(String args[])
{
```

```

        try
        {
            //below code do not throw any exception
int data=25/5;
            System.out.println(data);
        }
        //catch won't be executed
            catch(NullPointerException e)
        {
            System.out.println(e);
        }
        //executed regardless of exception occurred or not
        finally
        {
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    } }

```

Java throw keyword

The Java throw keyword is used to throw an exception explicitly.

```

public class TestThrow
{
    //function to check if person is eligible to vote or not

    public static void validate(int age)
    {
        if(age<18)
        {
            //throw Arithmetic exception if not eligible to vote

            throw new ArithmeticException("Person is not eligible to vote");

```

```

    }
else
{
    System.out.println("Person is eligible to vote!!");
}
}

//main method
public static void main(String args[])
{
    //calling the function
    TestThrow.validate(13);
    System.out.println("rest of the code...");
}
}

```

User defined Exception

You can also create your own exception sub class simply by extending java Exception class. You can define a constructor for your Exception sub class (not compulsory) and you can override the toString()function to display your customized message on catch.

```

class NegativeNumberException extends Exception
{
    public NegativeNumberException (String str)
    {
        // Calling constructor of parent Exception
        super(str);
    }
}

public class TestUser
{
    public static void main(String args[])
    {

```

```
try
{
    int n = Integer.parseInt(args[0]);
    if (n<0)
    {
        // throw an object of user defined exception
        throw new NegativeNumberException (n+" is negative");
    }
}
catch (NegativeNumberException e)
{
    System.out.println("Caught the exception");
    System.out.println(e.getMessage());
}
}
}
```

Java I/O

Java I/O (Input and Output) is used to process the input and produce the output. Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

Stream

A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

- 1) **System.out**: standard output stream
- 2) **System.in**: standard input stream
- 3) **System.err**: standard error stream

String class

In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string. This class present in the package java.lang which contains two string classes:

- a) String class
- b) StringBuffer class

Java String class methods

The java.lang.String class provides many useful methods to perform operations on sequence of char values.

Method	
char charAt(int index)	It returns char value for the particular index
int length()	It returns string length
static String format(String format, Object... args)	It returns a formatted string.
static String format(Locale l, String format, Object... args)	It returns formatted string with given locale.
String substring(int beginIndex)	It returns substring for given begin index.
String substring(int beginIndex, int endIndex)	It returns substring for given begin index and end index.
boolean contains(CharSequence s)	It returns true or false after matching the sequence of char value.
static String join(CharSequence delimiter, CharSequence... elements)	It returns a joined string.
boolean isEmpty()	It checks if string is empty.
String concat(String str)	It concatenates the specified string.
String replace(char old, char new)	It replaces all occurrences of the

	specified char value.
String replace(CharSequence old, CharSequence new)	It replaces all occurrences of the specified CharSequence.
static String equalsIgnoreCase(String another)	It compares another string. It doesn't check case.
String[] split(String regex)	It returns a split string matching regex.
String[] split(String regex, int limit)	It returns a split string matching regex and limit.
String intern()	It returns an interned string.
int indexOf(int ch)	It returns the specified char value index.
int indexOf(int ch, int fromIndex)	It returns the specified char value index starting with given index
int indexOf(String substring)	It returns the specified substring index.
int indexOf(String substring, int fromIndex)	It returns the specified substring index starting with given index.
String toLowerCase()	It returns a string in lowercase.
String toUpperCase()	It returns a string in uppercase.
String trim()	It removes beginning and ending spaces of this string.
static String valueOf(int value)	It converts given type into string. It is an overloaded method.

String Buffer Class

StringBuffer class in Java represents a sequence of characters that can be modified, which means we can change the content of the StringBuffer without creating a new object every time. It represents a mutable sequence of characters.

1. Unlike String, we can modify the content of the StringBuffer without creating a new object.
2. All methods of StringBuffer are synchronized, making it safe to use in multithreaded environments.
3. Ideal for scenarios with frequent modifications like append, insert, delete or replace

Constructors of StringBuffer Class

1. StringBuffer(): It reserves room for 16 characters without reallocation.
2. StringBuffer(int size): It accepts an integer argument that explicitly sets the size of the buffer.

3. **StringBuffer(String str):** It accepts a string argument that sets the initial contents of the StringBuffer object and reserves room for 16 more characters without reallocation.

String Buffer Methods

Sr.No.	Method & Description
1	<u>StringBuffer append()</u> This method appends the given string argument to the sequence.
2	<u>StringBuffer appendCodePoint(int codePoint)</u> This method appends the string representation of the codePoint argument to this sequence.
3	<u>int capacity()</u> This method returns the current capacity.
4	<u>char charAt(int index)</u> This method returns the char value in this sequence at the specified index.
5	<u>int codePointAt(int index)</u> This method returns the character (Unicode code point) at the specified index.
6	<u>int codePointBefore(int index)</u> This method returns the character (Unicode code point) before the specified index.
7	<u>int codePointCount(int beginIndex, int endIndex)</u> This method returns the number of Unicode code points in the specified text range of this sequence.
8	<u>StringBuffer delete(int start, int end)</u> This method removes the characters in a substring of this sequence.
9	<u>StringBuffer deleteCharAt(int index)</u> This method removes the char at the specified position in this sequence.
10	<u>void ensureCapacity(int minimumCapacity)</u> This method ensures that the capacity is at least equal to the specified minimum.
11	<u>void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)</u> Characters are copied from this sequence into the destination character array dst.

12	<u>int indexOf(String str)</u> This method returns the index within this string of the first occurrence of the specified substring.
13	<u>StringBuffer insert()</u> This method inserts the string representation of the given argument into this sequence.
14	<u>int lastIndexOf(String str)</u> This method returns the index within this string of the rightmost occurrence of the specified substring.
15	<u>int length()</u> This method returns the length (character count).
16	<u>int offsetByCodePoints(int index, int codePointOffset)</u> This method returns the index within this sequence that is offset from the given index by codePointOffset code points.
17	<u>StringBuffer replace(int start, int end, String str)</u> This method replaces the characters in a substring of this sequence with characters in the specified String.
18	<u>StringBuffer reverse()</u> This method causes this character sequence to be replaced by the reverse of the sequence.
19	<u>void setCharAt(int index, char ch)</u> Character at the specified index is set to ch.
20	<u>void setLength(int newLength)</u> This method sets the length of the character sequence.
21	<u>CharSequence subSequence(int start, int end)</u> This method returns a new character sequence that is a subsequence of this sequence.
22	<u>String substring(int start)</u> This method returns a new String that contains a subsequence of characters currently contained in this character sequence.
23	<u>String toString()</u> This method returns a string representing the data in this sequence.
24	<u>void trimToSize()</u> This method attempts to reduce storage used for the character sequence.

Java File Class

The File class is an abstract representation of file and directory pathname. A pathname can be either absolute or relative.

The File class have several methods for working with directories and files such as creating new directories or files, deleting and renaming directories or files, listing the contents of a directory

Sr.No.	Method & Description
1	File(File parent, String child) This constructor creates a new File instance from a parent abstract pathname and a child pathname string.
2	File(String pathname) This constructor creates a new File instance by converting the given pathname string into an abstract pathname.
3	File(String parent, String child) This constructor creates a new File instance from a parent pathname string and a child pathname string.
4	File(URI uri) This constructor creates a new File instance by converting the given file: URI into an abstract pathname.

File class methods

Sr.No.	Method & Description
1	getName() Returns the name of the file or directory.
2	getAbsolutePath() Returns the absolute pathname of file.
3	canRead() Tests whether the application can read the file denoted by this abstract pathname. Returns true if and only if the file specified by this abstract pathname exists and can be read by the application; false otherwise.
4	canWrite() Tests whether the application can modify to the file denoted by this abstract pathname. Returns true if and only if the file system actually contains a file denoted by this abstract pathname and the application is allowed to write to the file; false otherwise.
5	exists() Tests whether the file or directory denoted by this abstract pathname exists. Returns true if and only if the file or directory denoted by this abstract pathname exists; false otherwise.

6	isDirectory() Tests whether the file denoted by this abstract pathname is a directory. Returns true if and only if the file denoted by this abstract pathname exists and is a directory; false otherwise.
7	isFile() Tests whether the file denoted by this abstract pathname is a normal file. A file is normal if it is not a directory and, in addition, satisfies other system-dependent criteria. Any non-directory file created by a Java application is guaranteed to be a normal file. Returns true if and only if the file denoted by this abstract pathname exists and is a normal file; false otherwise.
8	length() Returns the size of the file in bytes.
9	createNewFile() Atomically creates a new, empty file. Returns true if the named file does not exist and was successfully created; false if the named file already exists.
10	delete() Deletes the file or directory denoted by this abstract pathname. If this pathname denotes a directory, then the directory must be empty in order to be deleted. Returns true if and only if the file or directory is successfully deleted; false otherwise.
11	list() Returns an array of strings naming the files and directories in the directory.
12	mkdir() Creates the directory. Returns true if and only if the directory was created; false otherwise.

Java FileInputStream Class

Java FileInputStream class obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video etc. You can also read character-stream data. But, for reading streams of characters, it is recommended to use FileReader class.

Java FileOutputStream Class

Java FileOutputStream is an output stream used for writing data to a file. If you have to write primitive values into a file, use FileOutputStream class. You can write byte-oriented as well as character-oriented data through FileOutputStream class. But, for character-oriented data, it is preferred to use FileWriter than FileOutputStream.

Java FileReader Class

Java FileReader class is used to read data from the file. It returns data in byte format like FileInputStream class. It is character-oriented class which is used for file handling in java.

Sr.No.	Constructor & Description
1	FileReader(File file) This constructor creates a new FileReader, given the File to read from.
2	FileReader(FileDescriptor fd) This constructor creates a new FileReader, given the FileDescriptor to read from.
3	FileReader(String fileName) This constructor creates a new FileReader, given the name of the file to read from.

Methods of FileReader class

Sr.No.	Method & Description
1	int read() Reads a single character. Returns an int, which represents the character read.
2	int read(char [] c, int offset, int len) Reads characters into an array. Returns the number of characters read.

Java FileWriter Class

Java FileWriter class is used to write character-oriented data to a file. It is character-oriented class which is used for file handling in java.

Unlike FileOutputStream class, you don't need to convert string into byte array because it provides method to write string directly.

Sr.No.	Constructor & Description
1	FileWriter(File file) This constructor creates a FileWriter object given a File object.
2	FileWriter(FileDescriptor fd) This constructor creates a FileWriter object associated with the given file descriptor.
3	FileWriter(String fileName) This constructor creates a FileWriter object, given a file

	name.
--	-------

Methods of FileWriter class

Sr.No.	Method & Description
1	void write(int c) This method writes a single character.
2	void write(char [] c, int offset, int len) This method writes a portion of an array of characters starting from offset and with a length of len.
3	void write(String s, int offset, int len) This method writes a portion of a String starting from offset and with a length of len.

Java DataInputStream Class

Java DataInputStream class allows an application to read primitive data from the input stream in a machine-independent way.

Java application generally uses the data output stream to write data that can later be read by a data input stream.

Java DataInputStream class Methods

Methods	Discription
int read(byte[] b)	to read the number of bytes from the input stream.
int read(byte[] b, int off, int len)	to read len bytes of data from the input stream.
int readInt()	to read input bytes and return an int value.
byte readByte()	to read and return the one input byte.
char readChar()	to read two input bytes and returns a char value.
double readDouble()	to read eight input bytes and returns a double value.
boolean readBoolean()	to read one input byte and return true if byte is non zero, false if byte is zero.
int skipBytes(int x)	to skip over x bytes of data from the input stream.
String readUTF()	to read a string that has been encoded using the UTF-8 format.
void readFully(byte[] b)	to read bytes from the input stream and store them into the buffer array.
void	to read len bytes from the input stream.

readFully(byte[] b, int off, int len)	
--	--

Java DataOutputStream Class

Java DataOutputStream class allows an application to write primitive Java data types to the output stream in a machine-independent way. Java application generally uses the data output stream to write data that can later be read by a data input stream.

Java DataOutputStream class methods :

Sr.No.	Method & Description
1	void flush() This method flushes this data output stream.
2	int size() This method returns the current value of the counter written, the number of bytes written to this data output stream so far.
3	void write(byte[] b, int off, int len) This method writes len bytes from the specified byte array starting at offset off to the underlying output stream.
4	void write(int b) This method writes the specified byte (the low eight bits of the argument b) to the underlying output stream.
5	void writeBoolean(boolean v) This method writes a boolean to the underlying output stream as a 1-byte value.
6	void writeByte(int v) This method writes out a byte to the underlying output stream as a 1-byte value.
7	void writeBytes(String s) This method writes out the string to the underlying output stream as a sequence of bytes.
8	void writeChar(int v) This method writes a char to the underlying output stream as a 2-byte value, high byte first.
9	void writeChars(String s) This method writes a string to the underlying output stream as a sequence of characters.

Lab Assignment

SET A

1. Write a java program to accept a number from the user, if number is zero then throw user defined exception —Number is 0, otherwise check whether no is prime or not.
2. Write a java program that displays the number of characters, lines and words of a file.
3. Write a program to copy the contents from one file into another file in upper case.
4. Write a class Student with attributes roll no, name, age and course. Initialize values through parameterized constructor. If age of student is not in between 15 and 21 then generate user-defined exception —Age Not Within The Range.

SET B

1. Write a java program to accept Doctor Name from the user and check whether it is valid or not. (It should not contain digits and special symbol) If it is not valid then throw user defined Exception - Name is Invalid -- otherwise display it.
2. Define a class MyDate (day, month, year) with methods to accept and display MyDate object. Accept date as dd,mm, yyyy. Throw user defined exception “InvalidDateException” if the date is invalid. Examples of invalid dates : 12 15 2015, 31 6 1990, 29 2 2001
3. Write a program to accept details of 'n' customers (c_id, cname, address, mobile_no) from user and store it in a file using DataOutputStream class.

SET C

1. Define a class SavingAccount (acno, name, balance). Define appropriate operations as, withdraw(), deposit(), and viewbalance(). The minimum balance must be 500. Create an object and perform operation. Raise user defined —InsufficientFundException when balance is not sufficient for withdraw operation.

Signature of the instructor: _____ Date: _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Interface Name	Description
Collection	This enables you to work with groups of objects; it is at the top of the collection's hierarchy.
List	This extends Collection and an instance of List stores an ordered collection of elements.
Set	This extends Collection to handle sets, which must contain unique elements.
SortedSet	This extends Set to handle sorted sets.
Map	This maps unique keys to values.
SortedMap	This extends Map so that the keys are maintained in an ascending order.

Collection Interface methods

Method	Description
boolean add(Object obj)	Adds obj to the invoking collection.
boolean addAll(Collection c)	Adds all the elements of c to the invoking collection.
void clear()	Removes all elements from the invoking collection.
boolean contains(Object obj)	Returns true if obj is an element of the invoking collection.
boolean containsAll(Collecti	Returns true if the invoking collection contains all elements of c.

on c)	
boolean equals(Object obj)	Returns true if the invoking collection and obj are equal.
int hashCode()	Returns the hash code for the invoking collection.
boolean isEmpty()	Returns true if the invoking collection is empty.
Iterator iterator()	Returns an iterator for the invoking collection.
boolean remove(Object obj)	Removes one instance of obj from the invoking collection. Returns true if the element was removed.
boolean removeAll(Collection c)	Removes all elements of c from the invoking collection.
boolean retainAll(Collection c)	Removes all elements from the invoking collection except those in c.
int size()	Returns the number of elements held in the invoking collection.
Object[] toArray()	Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection elements.
Object[] toArray(Object array[])	Returns an array containing only those collection elements whose type matches that of array.

List Interface

List is a child interface of Collection. It allows to store heterogeneous data.

- Here duplicates are allowed and insertion order is preserved
- Implemented class of List are
ArrayList, LinkedList, Vector (legacy classes)

Apart from methods of Collection Interface, it adds following methods of its own.

Method	Description
void add(int index, Object obj)	Inserts obj into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up.
boolean addAll(int index, Collection c)	Inserts all elements of c into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up.
Object get(int index)	Returns the object stored at the specified index within the invoking collection.
int indexOf(Object obj)	Returns the index of the first instance of obj in the invoking list. If obj is not an element of the list, -1 is returned.
int lastIndexOf(Object obj)	Returns the index of the last instance of obj in the invoking list. If obj is not an element of the list, -1 is returned.
ListIterator listIterator()	Returns an iterator to the start of the invoking list.
ListIterator listIterator(int index)	Returns an iterator to the invoking list that begins at the specified index.
Object remove(int index)	Removes the element at position index from the invoking list and returns the deleted element.
Object set(int index, Object obj)	Assigns obj to the location specified by index within the invoking list.
List subList(int start, int end)	Returns a list that includes elements from start to end-1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

Set Interface and SortedSet Interface

Set is a child interface of Collection.

- This interface defines a Set.
- Set Interface and SortedSet Interface is a collection that contains no duplicate elements
- Implemented class of Set are HashSet, LinkedHashSet, TreeSet.
- Duplicates are not allowed and insertion order is not preserved.
- It contains the methods of the Collection interface(add (), clear(),contains() etc).
- A SortedSet is a Set that maintains its elements in ascending order. It allows to store homogeneous data..

Method	Description
Object first()	Returns the first element in the invoking sorted set.
Object last()	Returns the last element in the invoking sorted set.

Comparator comparator()	Returns the invoking sorted set's comparator. If the natural ordering is used for this set, null is returned.
SortedSet headSet(Object end)	Returns a SortedSet containing those elements less than end that are contained in the invoking sorted set. Elements in the returned sorted set are also referenced by the invoking sorted set.
SortedSet subSet(Object start, Object end)	Returns a SortedSet that includes those elements between start and end.1. Elements in the returned collection are also referenced by the invoking object.
SortedSet tailSet(Object start)	Returns a SortedSet that contains those elements greater than or equal to start that are contained in the sorted set. Elements in the returned set are also referenced by the invoking object.

Queue Interface

- It extends collection interface and defines behaviour of queue, that is first-in, first-out.
- Queue(I) is a child interface of Collection.

Method	Description
Object poll()	removes element at the head of the queue and returns null if queue is empty
Object remove()	removes element at the head of the queue and throws NoSuchElementException if queue is empty
Object peek()	returns the element at the head of the queue without removing it. Returns null if queue is empty
Object element()	same as peek(), but throws NoSuchElementException if queue is empty
booleanoffer(E obj)	Adds object to queue.

Map Interface:

- A map contains values on the basis of key, i.e. key and value pair. Each key and value pair is known as an entry. A Map contains unique keys.

- A Map is useful if you have to search, update or delete elements on the basis of a key.

Method	Description
void clear()	Removes all key/value pairs from the invoking map.
boolean containsKey(Object k)	Returns true if the invoking map contains k as a key.
boolean containsValue(Object v)	Returns true if the map contains v as a value.
Set entrySet()	Returns a Set that contains the entries in the map. The set contains objects of type Map.Entry. This method provides a set-view of the invoking map.
boolean equals(Object obj)	Returns true if obj is a Map and contains the same entries.
Object get(Object k)	Returns the value associated with the key k.
int hashCode()	Returns the hash code for the invoking map.
boolean isEmpty()	Returns true if the invoking map is empty.
Set keySet()	Returns a Set that contains the keys in the invoking map. This method provides a set-view of the keys in the invoking map.
Object put(Object k, Object v)	Puts an entry in the invoking map, overwriting any previous value associated with the key.
void putAll(Map m)	Puts all the entries from m into this map.
Object remove(Object k)	Removes the entry whose key equals k.
int size()	Returns the number of key/value pairs in the map.
Collection values()	Returns a collection containing the values in the map. This method provides a collection-view of the values in the map

List Implementation

- Duplicate are allowed in list. List allows any no of null values. The Storage Type of list is index. List maintained order of insertion.
- There are following general-purpose List implementations.

1. Arraylist

1. duplicate are allowed in Arraylist
2. allows any no of null values
3. storage type: index
4. order of insertion-maintain
5. Default capacity for arraylist is 10
6. data structure: Resizable
7. Incremental capacity= $(\text{current capacity} * 3/2) + 1$

8. best choice: retrieval operation (random access interface is implemented in arraylist & vector)
9. worst choice: manipulation operation i.e. insertion in between arraylist or delete ()

2. LinkedList -The LinkedList class implements the List interface.

1. Duplicate are allowed in LinkedList
2. allows any no of null values
3. order of insertion-maintain
4. storage type: index
5. No Default capacity in linkedlist
6. data structure: linear
7. best choice: manipulation operation i.e. insertion in between linkedlist or delete()
8. worst choice: retrieval operation (random access interface is not implemented)

3. Vector: Implements Dynamic array.

1. duplicate are allowed in vector
2. allows any no of null values
3. storage type: index
4. order of insertion-maintain
5. Default capacity for Vector is 10
6. data structure: doubly
7. Incremental capacity= current capacity*2
8. best choice: retrieval operation (random access interface is implemented in arraylist & vector)
9. worst choice: manipulation operation i.e. insertion in between Vector or delete()
10. Vector is legacy class.

Vector Constructors:

1. Vector() : Default Vector with initial size zero.
2. Vector(int size):Vector with initial capacity as size.
3. Vector(int size, int increment): Constructs a vector whose initial capacity is specified b size and whose increment is specified by incr.
4. Vector(Collection c):Construct a vector that contains the elements of collection c.

Set Implementation

Set doesn't allow duplicate . order of insertion is random insertion

There are three general-purpose Set implementations — HashSet, TreeSet and LinkedHashSet.

1. Hashset:

1. Doesn't allow duplicate values
2. Allow only 1 null value.
3. order of insertion-random insertion
4. storage type: hashtable
5. no default capacity
6. Data Structure: Hashtable
7. Best choice: To remove duplicate elements when order of insertion is not mandatory.

2. LinkedHashSet:

1. Doesn't allow duplicate values
2. Allow only 1 null value.
3. order of insertion-maintained
4. no default capacity
5. Data Structure: Hybrid (liner+ hashtable)
6. storage type: hashtable
7. Best choice: To remove duplicate elements when order of insertion is mandatory

3. TreeSet:

1. doesn't allow duplicate
2. null values: not allowed
3. order of insertion- Asscending order.
4. no default capacity.
5. Data Structure : Balanced tree
6. storage type: hashtable

Map Implemetation

There are three general-purpose Map implementations — HaspMap, LinkedHashMap,

TreeMap

1. **HashMap:** The HashMap class uses a hashtable to implement the Map interface. This allows the execution time of basic operations, such as get() and put(), to remain constant even for large sets.

2. **LinkedHashMap:**

This class extends HashMap and maintains a linked list of the entries in the map, in the order in which they were inserted. This allows insertion-order iteration over the map. That is, when iterating a LinkedHashMap, the elements will be returned in the order in which they were inserted.

2. **TreeMap:**

The TreeMap class implements the Map interface by using a tree. A TreeMap provides an efficient means of storing key/value pairs in sorted order, and allows rapid retrieval.

Cursors in Collection

1. **Iterator(I)**

1. **Iterator** The Iterator interface provides methods using which we can traverse any collection. This interface is implemented by all collection classes.
2. Using iterator we can traverse collection object only in forward direction.
3. By using iterator we can perform only read and remove operations and we can not perform replace and addition of new object.

Methods by Iterator

Methods	Description
boolean hasNext()	Returns true if there are more elements. Otherwise, returns false.
Object next()	Returns the next element. Throws NoSuchElementException if there is not a next element.
void remove()	Removes the current element. Throws IllegalStateException if an attempt is made to call remove() that is not preceded by a call to next().

2. **list Iterator(I)**

1. Only applicable for list interface type implementation classes
2. Using list iterator we can traverse a List in forward direction and backward direction that is it is bidirectional cursor
3. By using list Iterator we can perform read, remove, replace and addition of new object operations

Methods by ListIterator

Methods	Description
boolean hasNext()	Returns true if there is a next element. Otherwise, returns false.
boolean hasPrevious()	Returns true if there is a previous element. Otherwise, returns false.
Object next()	Returns the next element. A NoSuchElementException is thrown if there is not a next element.
int nextIndex()	Returns the index of the next element. If there is not a next element, returns the size of the list.
Object previous()	Returns the previous element. A NoSuchElementException is thrown if there is not a previous element.
int previousIndex()	Returns the index of the previous element. If there is not a previous element, returns - 1.
void remove()	Removes the current element from the list. An IllegalStateException is thrown if remove() is called before next() or previous() is invoked.
void set(Object obj)	Assigns obj to the current element. This is the element last returned by a call to either next() or previous().

3.Enumeration(I)

We can use Enumeration to get objects one by one from the legacy collection objects. We can create Enumeration object by using elements() method.

Enumeration interface defines the following two methods public boolean hasMoreElements(); public Object nextElement();

Comparator

- Comparator interface is used to order the objects of a user-defined class.
- This interface is found in java.util package and contains 2 methods compare(Object obj1, Object obj2) and equals(Object element).
- It provides multiple sorting sequences, i.e., you can sort the elements on the basis of any data member, for example, rollno, name, age or anything else

Methods of Java Comparator Interface

There are two methods of Comparators in, namely:

Methods	Description
compare(Object obj1, Object obj2)	Compares the first object with another
equals(Object obj)	Compares current object with specified obj

Sample Program1 : Program to demonstrate ArrayList

```
import java.util.ArrayList;
public class ArrayListUse {

    public static void main(String[] args) {

        ArrayList al= new ArrayList();

        al.add("Tybca");//0
        al.add("May-21");//1
        al.add(90);//2
        al.add('A');//3
        al.add(99.99f);//4
        al.add("Tybca");//5
        al.add(null);//6
        al.add(null);//7

        // printing the ArrayList
        System.out.println(al);
    }
}
```

Sample Program2 : Program to demonstrate LinkedList

```

package Collection;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.ListIterator;

public class example3_Linkedlist {
    public static void main(String[] args) {

        LinkedList ll=new LinkedList();
        ll.add("abc");
        ll.add(100);
        ll.add('A');
        ll.add(65.5f);
        ll.add(100);
        ll.add(null);
        ll.add(null);

        System.out.println("-----print info in linkdlist using iterator cursor-----");
        Iterator itr = ll.iterator();
        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}

```

Sample Program3 : Program to demonstrate HashSet

```

package CollectionUse;
import java.util.HashSet;
import java.util.Iterator;
public class Hash_Set_Study {
    public static void main(String[] args) {

        HashSet h= new HashSet();
        h.add(100);
        h.add("Tybca");
        h.add('A');
        h.add(100);// added duplicate
        h.add(66.7f);
        h.add(null);
        h.add(null);
        System.out.println(h);
        System.out.println(h.size());
        System.out.println(h.contains('A'));
        System.out.println(h.isEmpty());
        System.out.println(h);
    }
}

```

```

System.out.println(h.remove('A'));
System.out.println(h);
Iterator it= h.iterator();
while(it.hasNext())
{
    System.out.println(it.next());
}

}

```

Sample Program4 : Program to demonstrate LinkedHashSet

```

package CollectionUse;
import java.util.Iterator;
import java.util.LinkedHashSet;

public class Linked_HashSet_study {

    public static void main(String[] args) {
        LinkedHashSet lh= new LinkedHashSet();

        lh.add(100);
        lh.add("Tybca");
        lh.add('A');
        lh.add(null);
        lh.add(100);
        lh.add(null);
        lh.add('B');

        System.out.println(lh);
        System.out.println(lh.isEmpty());
        System.out.println(lh.size());
        System.out.println(lh.contains('B'));
        System.out.println(lh);
        lh.remove(100);
        System.out.println(lh);
        Iterator it= lh.iterator();
        while(it.hasNext())
        {
            System.out.println(it.next());
        }
    }
}

```

```
}
```

Sample Program5 :Program to demonstrate TreeSet

```
package CollectionUse;
import java.util.Iterator;
import java.util.TreeSet;
public class TreeSetExample {

    public static void main(String[] args) {
        TreeSet ts= new TreeSet();

        ts.add("xyz");
        ts.add("Tybca");
        ts.add("abc");
        ts.add("pune");
        ts.add("xyz");
        ts.add("XXX");
        ts.add("aaa");
        ts.add("AAA");
        System.out.println(ts);

        System.out.println(ts);
        Iterator it= ts.iterator();

        while(it.hasNext())
        {
            System.out.println(it.next());
        }
    }
}
```

Sample Program6 :Program to demonstrate HashTable

```
import java.util.*;
class Demo { public static void main(String args[])
{ // Creating Hashtable
    Hashtable hashtable = new Hashtable();
    // Adding elements
```

```

hashtable.put("a",100);
hashtable.put("b",200);
hashtable.put("c",300);
hashtable.put("d",400);
// Displaying Hashtable System.out.println(hashtable);
// Search for a value
booleanval = hashtable.contains(400);
System.out.println("is 400 present: "+val);
// Search for a key
val = hashtable.containsKey("d");
System.out.println("is d present: "+val);
} }

```

Sample Program7 :Program to demonstrate Queue

```

package CollectionUse;
import java.util.PriorityQueue;
public class PriorityQStudy {

    public static void main(String[] args) {

        PriorityQueue pq= new PriorityQueue();

        pq.add("abc");
        pq.add("pqr");
        pq.add("lmn");
        pq.add("xyz");
        pq.add("aaa");
        //pq.add(100);

        System.out.println(pq);
        System.out.println(pq.element());
        System.out.println(pq.poll());
        System.out.println(pq);
        System.out.println(pq.peek());
        System.out.println(pq);
        pq.poll();
        System.out.println("remove method "+pq.remove());
        pq.poll();
        pq.poll();
        System.out.println(pq);
    }
}

```

```
}
```

Sample Program 8 : Program to demonstrate Iterator

```
import java.util.ArrayList;
import java.util.Iterator;
public class Test { public static void main(String[] args)
{ ArrayList al = new ArrayList();
for (int i = 0; i < 10; i++)
al.add(i);
System.out.println(al);
// at beginning itr(cursor) will point to
// index just before the first element in al
Iterator itr = al.iterator();
// checking the next element availability
while (itr.hasNext())
{ // moving cursor to next element
int i = (Integer)itr.next();
// getting even elements one by one
System.out.print(i + " ");
// Removing odd elements
if (i % 2 != 0)
itr.remove();
}
System.out.println();
System.out.println(al);
} }
```

Lab Assignment

SET A

1. Create a Hash table containing Employee name and Salary. Display the details of the hash table. Also search for a specific Employee and display Salary of that Employee.
2. Write a java program to accept names of 'n' cities, insert same into array list collection and display the contents of same array list, also remove all these elements.
3. Write a java program to read 'n' names of your friends, store it into linked list, also display contents of the same.
4. Write a java program to read 'n' names of your friends, store it into linked list, also display contents of the same.

SET B

1. Accept n integers from the user and store them in a collection. Display them in the sorted order. The collection should not accept duplicate elements. (Use a suitable collection). Search for a particular element using predefined search method in the Collection framework.
2. Accept 'n' integers from the user. Store and display integers in sorted order having proper collection class. The collection should not accept duplicate elements.
3. Write a program to sort HashMap by keys and display the details before sorting and after sorting.

SET C

1. Construct a linked List containing names of colours: red, blue, yellow and orange. Then extend your program to do the following:
 - i. Display the contents of the List using an Iterator
 - ii. Display the contents of the List in reverse order using a ListIterator
 - iii. Create another list containing pink and green. Insert the elements of this list between blue and yellow.
2. Write a program to create link list of integer objects. Do the following:
 - i. add element at first position
 - ii. delete last element
 - iii. display the size of link list

Signature of the instructor: _____Date: _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []

Assignment No.	5	No. of Sessions:	3
Assignment Name:	Applet Programming and Multithreading		

Objective:

- To learn the user interface designing Java Programming.
- To write Java Program using applet programming and multithreading.

Reading:

- Object-Oriented Programming (OOP) Concepts
- Applet programming depends heavily on Graphics class.

Understanding:

Introduction to Applets

An **Applet** is a small Java program that runs inside a browser or an applet viewer. It is used to create dynamic and interactive web applications.

- Applets do not use main() method.
- They extend the Applet class (java.applet.Applet) or JApplet.
- Execution is controlled by the browser or viewer.

Applet Life Cycle

The applet lifecycle consists of five important methods:

Method	Description
init()	Initializes applet (called once)
start()	Called when applet starts/resumes
paint(Graphics g)	Used for rendering output
stop()	Called when applet is paused
destroy()	Cleanup before termination

Applet Execution Flow

1. Applet is loaded

2. init() is called
3. start() is called
4. paint() executes repeatedly
5. stop() when minimized
6. destroy() when closed

Basic Applet Program Structure

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class MyApplet extends Applet {
```

```
    public void init() {
```

```
        // initialization
```

```
    }
```

```
    public void paint(Graphics g) {
```

```
        g.drawString("Hello Applet", 50, 50);
```

```
    }
```

```
}
```

Graphics in Applet

The Graphics class is used to draw shapes and text.

Common methods:

- drawString(String, x, y)
- drawLine(x1, y1, x2, y2)
- drawRect(x, y, width, height)
- drawOval(x, y, width, height)

- fillRect(), fillOval()

Event Handling in Applets

Used to handle user interactions like mouse clicks or keyboard input.

- Event Delegation Model
- Interfaces:
 - MouseListener
 - KeyListener
 - ActionListener

Example methods:

- mouseClicked()
- mousePressed()

Limitations of Applets

- Security restrictions (sandbox)
- Browser support removed
- Replaced by modern technologies

Multithreading in Java

Introduction to Multithreading

Multithreading allows multiple threads to run concurrently within a program.

Advantages:

- Efficient CPU utilization
- Faster execution
- Improved responsiveness

Thread Creation Methods

Method 1: Extending Thread Class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}
```

```
}
```

Method 2: Implementing Runnable

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}
```

Thread Life Cycle

Stages:

1. New
2. Runnable
3. Running
4. Waiting / Blocked
5. Dead

Important Thread Methods

Method	Description
start()	Starts thread
run()	Thread logic
sleep()	Pauses thread
join()	Waits for another thread
isAlive()	Checks status

Synchronization

Used to control access to shared resources.

```
synchronized void display() {
```

```
// critical section
```

```
}
```

Prevents:

- Race condition
- Data inconsistency

Inter-thread Communication

Methods:

- wait()
- notify()
- notifyAll()

Used for coordination between threads.

Lab Assignment

SET A

1. Write a java program to create a simple applet that displays a message.
2. Write a java program to draw rectangle and oval.
3. Write a java program to handle mouse click event.
4. Write a java program to create thread by extending class.

SET B

1. Write a java program to create using Runnable interface to display traffic signal.
2. Write a java program to show Multiple Threads Execution

SET C

1. Write a java program to demonstrate Synchronization Example

Signature of the instructor: _____ **Date:** _____

Assignment Evaluation

0: Not Done []

1: Incomplete []

2: Late Complete []

3: Needs Improvement []

4: Complete []

5: Well done []